Graph RAG

Knowledge Graph

학습 로드맵

01 그래프 DB 기초

RDB vs 그래프 DB
LPG 개념

02 Cypher 문법

CREATE / MATCH
SET / DELETE / MERGE

03 지식그래프

온톨로지 · 트리플
추론(Inference)

04 스키마 전환

RDB → 그래프
전환 전략

05 LLM 추출

텍스트 → 엔티티
Neo4j 적재

06 Graph RAG

Text2Cypher
그래프 기반 답변

01. RDB vs 그래프 DB

관계형 DB (RDB)

- 테이블 구조로 데이터 저장
- 관계는 외래키(FK)로 표현
- 조회 시 JOIN 필요

SQL 예시:

```
SELECT s.name FROM ship s
JOIN block b ON s.id=b.ship_id
JOIN process p ON b.id=p.block_id
JOIN worker w ON p.id=w.process_id
WHERE w.name='김철수'
```

VS

그래프 DB (Neo4j)

- 노드와 관계로 데이터 저장
- 관계를 직접 표현 (화살표)
- 관계 탐색이 직관적.빠름

Cypher 예시:

```
MATCH
  (w:Worker {name:'김철수'})
  -[:EXECUTES]->(p:Process)
  <-[:PROCESSED_BY]-(b:Block)
  -[:BELONGS_TO]->(s:Ship)
RETURN s.name
```

02. LPG (Labeled Property Graph) 구성요소

① 노드 (Node)

사물·개념을 표현하는 기본 단위
예) 블록, 선박, 작업자, 설비

```
(:Block)
(:Worker)
(:Ship)
```

② 레이블 (Label)

노드의 타입/분류를 지정
콜론(:)으로 구분. 복수 레이블 가능

```
(n:Worker)
(n:Worker:Certified)
```

③ 프로퍼티 (Property)

노드·관계에 붙는 key:value 속성
노드와 관계 양쪽에 모두 사용 가능

```
({name:'김철수',
grade:'특급'})
```

④ 관계 (Relationship)

노드 간 연결. 방향과 타입 필수
관계에도 프로퍼티 부여 가능

```
-[:EXECUTES
{date:'2025'}]->
```

03. Cypher 문법

CREATE

노드·관계 생성. 중복 무시하고 새로 생성

```
CREATE (b:Block {id:"B-101", status:"용접완료"})
-[:PROCESSED_BY {seq:3}]->
(p:Process {method:"SAW"})
```

MERGE

있으면 찾고, 없으면 생성. 중복 방지에 필수

```
MERGE (b:Block {id:"B-101"})
ON CREATE SET b.status = "대기중"
ON MATCH  SET b.status = "업데이트됨"
```

MATCH

패턴에 맞는 노드·관계 검색

```
MATCH (w:Worker)-[:EXECUTES]->(p:Process)
<-[:PROCESSED_BY]-(b:Block)
RETURN w.name, p.method, b.id
```

SET / DELETE

SET: 프로퍼티 수정 | DETACH DELETE: 노드 삭제

```
MATCH (b:Block {id:"B-101"})
SET b.status = "도장중"

MATCH (b:Block {id:"TEST"})
DETACH DELETE b
```

04. Cypher 쿼리

집계 — 작업자별 공정 수

```
MATCH (w:Worker)-[:EXECUTES]->(p:Process)
RETURN w.name, count(p) AS 공정수
ORDER BY 공정수 DESC
```

WITH — 조건 필터링

```
MATCH (s:Ship)-[:CONSISTS_OF]->(b:Block)
WITH s, count(b) AS 블록수
WHERE 블록수 >= 2
RETURN s.name, 블록수
```

OPTIONAL MATCH — null 허용 조회

```
MATCH (b:Block)
OPTIONAL MATCH (b)-[:PROCESSED_BY]->(p)
RETURN b.id, b.status, p.type
```

가변 길이 경로 탐색

```
MATCH (s:Ship {id:'LNG-001'})
      -[*1..3]->(n)
RETURN DISTINCT labels(n), n
```

05. 지식그래프 & 온톨로지

트리플(Triple) — 지식그래프의 기본 단위: 주어 - 술어 - 목적어

김철수	담당한다	용접공정 W-07
블록 B-101	속한다	LNG-001 선박
용접공정	사용한다	자동용접기
SAW 용접	종류이다	제조공정

온톨로지 — 스키마 규칙 정의

```
Ship  -[CONSISTS_OF]→  Block  -[PROCESSED_BY]→  Process  ←[EXECUTES]-  Worker
제약: Worker는 반드시 skill 프로퍼티 보유 | Process는 반드시 type 프로퍼티 보유
```

06. RDB → 그래프 스키마 전환 전략

핵심 전환 규칙

테이블	→	**노드 레이블 (:Block, :Worker)**
행 (Row)	→	**노드 (개별 인스턴스)**
외래키 (FK)	→	**관계 ([:PROCESSED_BY])**
JOIN 쿼리	→	**관계 탐색 (패턴 매칭)**

그래프 전환 권장 시점

권장	JOIN이 3개 이상 반복될 때
권장	관계가 자주 변경될 때
권장	계층구조·네트워크 데이터
유지	단순 집계·통계 중심
유지	트랜잭션 중심 업무

07. LLM 기반 Knowledge Extraction

텍스트 문서 → LLM 추출 → JSON 구조화 → Cypher 변환 → Neo4j 적재

입력 텍스트

"B-101 블록의 용접 작업은 김철수 작업자가
담당하며, SAW 방식으로 2도크에서 진행 중.
해당 블록은 LNG-001 선박 선수 구역."

LLM 추출 결과 (JSON)

```
entities: [
  {id:"B-101",  label:"Block"},
  {id:"김철수", label:"Worker"},
  {id:"LNG-001",label:"Ship"}
]
```

Python 적재 코드 (예시)

```python
driver = GraphDatabase.driver("neo4j://localhost:7687", auth=("neo4j","pw"))

def load_entities(tx, entities):
    for e in entities:
        tx.run(f"MERGE (n:{e['label']} {{id:$id}}) SET n += $props",
               id=e["id"], props=e["properties"])

with driver.session() as session:
    session.execute_write(load_entities, data["entities"])   # 노드 먼저
    session.execute_write(load_relationships, data["relationships"])  # 관계 나중
```

08. Graph RAG — 개념 및 흐름

벡터 RAG

텍스트를 벡터로 변환 후 유사도 검색

강점: 자연어 검색에 유리
약점: 관계 구조 파악 불가

예) '김철수 담당 블록' 검색 시
관련 문서만 반환 → 관계 추적 불가

Graph RAG

그래프 관계를 타고 정확한 데이터 검색

강점: 관계 추적 · 정확한 답변
약점: 스키마 설계 필요

예) 김철수→공정→블록→선박
단계별 연결을 정확히 탐색

Graph RAG 처리 흐름

1. 질문 입력	2. Text2Cypher	3. 그래프 탐색	4. 답변 생성
자연어 질문	LLM이 Cypher 생성	Neo4j 쿼리 실행	LLM이 자연어 답변

09. Graph RAG — Python 구현

```
from anthropic import Anthropic
from neo4j import GraphDatabase

client = Anthropic()
driver = GraphDatabase.driver("neo4j://localhost:7687", auth=("neo4j","password"))

# ① 자연어 → Cypher 변환
def text_to_cypher(question):
    resp = client.messages.create(
        model="claude-sonnet-4-20250514", max_tokens=500,
        system="""Neo4j 전문가. 질문을 Cypher로 변환.
스키마: Ship, Block, Process, Worker, Equipment
관계: CONSISTS_OF, PROCESSED_BY, EXECUTES, USED_IN
Cypher만 반환.""",
        messages=[{"role":"user","content": question}]
    )
    return resp.content[0].text

# ② 그래프 탐색
def query_graph(cypher):
    with driver.session() as session:
        return [r.data() for r in session.run(cypher)]

# ③ 자연어 답변 생성
def generate_answer(question, data):
    resp = client.messages.create(
        model="claude-sonnet-4-20250514", max_tokens=500,
        system="그래프 검색 결과를 바탕으로 자연어로 답변하세요.",
        messages=[{"role":"user","content":f"질문:{question}\n데이터:{data}"}]
    )
    return resp.content[0].text
```

10. 제조 적용 시나리오

Q. 크레인 CR-01이 고장나면 어떤 공정이 영향받나?

→ CR-01 → W-01(용접) → B-101, B-303 블록 작업 중단
담당자 김철수 즉시 통보 필요

Q. LNG-001 선박의 용접 공정 담당자 전체 목록은?

→ Ship→Block→Process←Worker 경로 탐색
김철수(B-101), 박민수(B-303) 반환

Q. 현재 '대기중' 상태인 블록과 배정된 작업자는?

→ MATCH (b:Block {status:'대기중'}) 조회
미배정 블록은 OPTIONAL MATCH로 null 처리

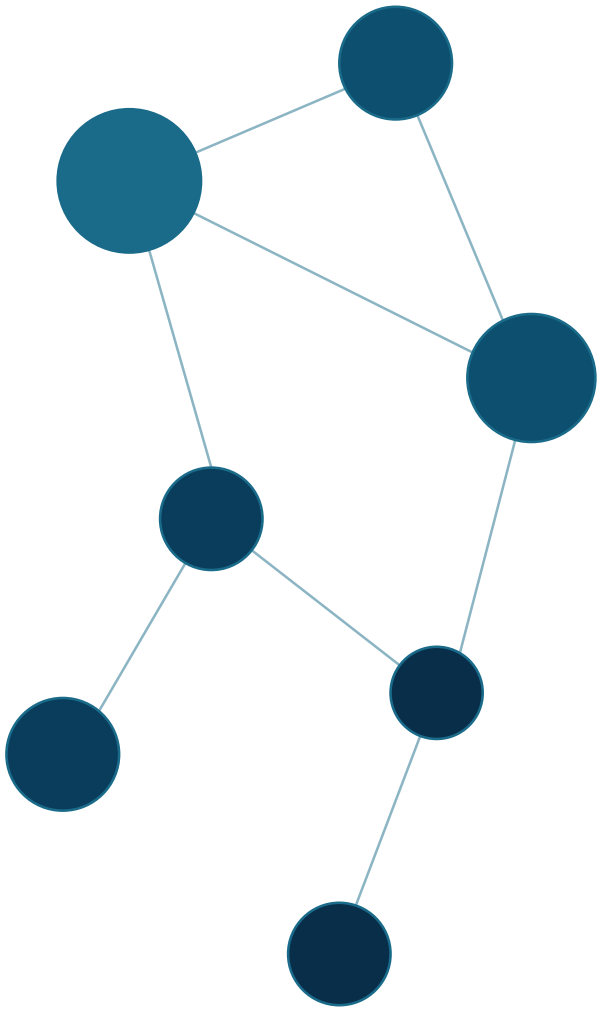
Q. 특급 기술자가 수행 가능한 모든 공정 유형은?

→ 온톨로지 추론: Worker(grade=특급) → 자격 규칙 적용
가변 경로 *1..2 탐색으로 공정 목록 추출

Neo4j · Graph RAG · 온톨로지

그래프 데이터베이스와 지식 표현의 이해

지식 그래프 기반 AI 검색 아키텍처



목 차

01 **온톨로지 (Ontology)**
개념, 클래스, 관계 — 지식의 구조화된 표현

02 **Neo4j & 그래프 데이터베이스**
노드, 엣지, 속성 — 그래프 모델의 이해와 Cypher 쿼리

03 **지식 그래프 (Knowledge Graph)**
RDF, 트리플, 온톨로지와 그래프 DB의 연계

04 **Graph RAG**
그래프 기반 검색 증강 생성 아키텍처

05 **실습 및 활용 사례**
GraphRAG 파이프라인 구현과 도메인 적용

01 온톨로지 (Ontology)

온톨로지는 특정 도메인 내의 개념(Concept), 개체(Entity), 관계(Relation)를 형식적으로 정의한 지식 표현 체계
철학에서 출발하여 AI/정보공학에서 지식 모델링의 핵심 도구로 사용

클래스 (Class)	속성 (Property)	관계 (Relation)
개념의 집합 EX) Person, Organization, Document, Event	클래스의 특성 기술 Datatype Property : 이름, 날짜 Object Property : 관계 연결	개체 간의 의미적 연결 isA, partOf, worksFor 상속 및 추론 지원

주요 표준 언어 및 형식

RDF

Resource Description Framework — 트리플 (Subject-Predicate-Object) 기반 데이터 모델

OWL

Web Ontology Language — 복잡한 클래스 관계, 제약 조건, 추론 규칙 정의

SPARQL

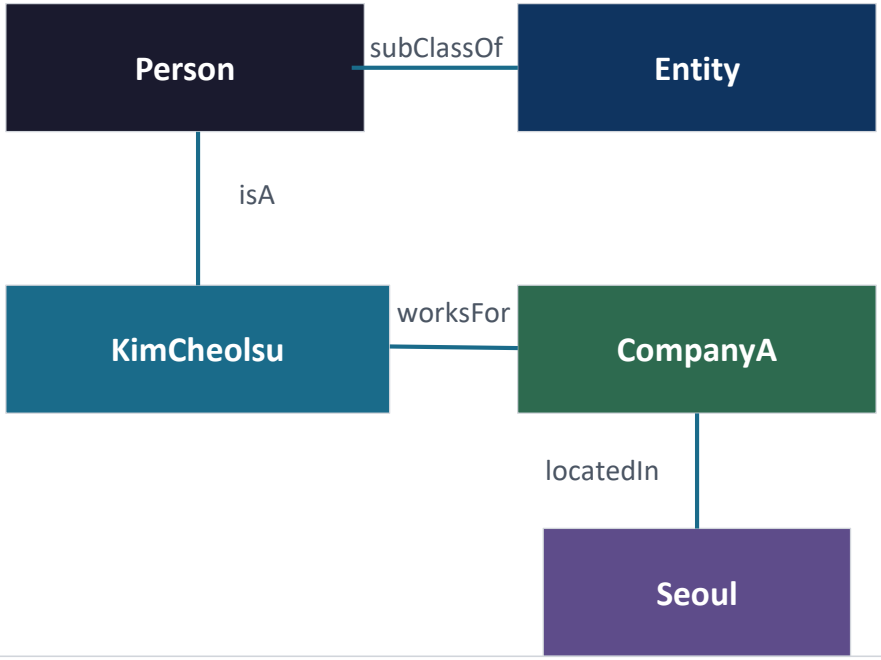
RDF 그래프 질의 언어 — SQL과 유사한 패턴 매칭 기반 쿼리

01 온톨로지 — 구조 예시

RDF 트리플 (Triple)

```
# 주어 - 술어 - 목적어

:KimCheolsu  rdf:type  :Person .

:KimCheolsu  :worksFor  :CompanyA .

:CompanyA  :locatedIn  :Seoul .

:Person  rdfs:subClassOf  :Entity .
```

시각적 그래프 표현



온톨로지는 도메인 지식을 기계가 이해 가능한 형태로 명시화하여 추론(Reasoning)과 지식 공유를 가능하게 합니다.

02 Neo4j & 그래프 데이터베이스

Property Graph Model

N

노드 (Node)

개체 표현. 레이블로 분류
예 : (:Person), (:Document)

R

관계 (Relationship)

방향이 있는 엣지. 유형 보유
예 : -[:KNOWS]->, -[:WROTE]->

P

속성 (Property)

키-값 쌍. 노드와 관계 모두 보유
예 : {name: '김철수', since: 2023}

Cypher 쿼리 언어

```
// 노드 생성
CREATE (:Person {name: '김철수',
        age: 32})

// 관계 생성
MATCH (a:Person {name:'김철수'}),
      (b:Company {name:'A사'})
CREATE (a)-[:WORKS_FOR {
  since: 2022}]->(b)

// 2홉 이내 동료 탐색
MATCH (p:Person)-[:KNOWS*1..2]
      ->(colleague)
RETURN colleague.name
```

관계형 DB 대비 강점: 다중 홉 탐색(수 ms), 유연한 스키마, 연결 중심 쿼리에 최적화

03 지식 그래프 (Knowledge Graph)

지식 그래프는 실세계 개체와 그 관계를 그래프 구조로 표현한 데이터베이스
Google Knowledge Graph, Wikidata, DBpedia 등이 대표 사례
온톨로지를 스키마로 활용

온톨로지 (Schema Layer)

- TBox — 용어 정의
- 클래스 계층 구조
- 관계 제약 조건
- 추론 규칙 정의

지식 그래프 (Instance Layer)

- ABox — 개체 데이터
- 실제 엔터티와 관계
- 속성 값 저장
- 대규모 팩트 베이스

Neo4j (Storage/Query)

- 그래프 저장 엔진
- Cypher 쿼리 인터페이스
- APOC 라이브러리
- 실시간 탐색·분석

대표 지식 그래프 사례

Google Knowledge Graph	Wikidata	DBpedia	Microsoft Satori	Amazon Product Graph

04 Graph RAG

기존 RAG의 한계와 Graph RAG의 등장

기존 RAG (Vector RAG)	Graph RAG
• 텍스트를 벡터 임베딩으로 변환 후 유사도 검색 • 지역적(local) 의미 검색에 강점 • 문서 간 관계·맥락 포착 어려움 • 다중 홉 추론 불가 • 전체 코퍼스 요약 질문에 취약	• 지식 그래프 기반 구조적 검색 결합 • 개체 간 관계·경로 활용 가능 • 커뮤니티 구조 기반 전역(global) 검색 • 다중 홉 추론 지원 • 출처 추적(traceability) 향상

Microsoft Research GraphRAG (2024):
LLM으로 지식 그래프를 자동 구축하고 커뮤니티 요약(Community Summary)을 생성하여 전역 질의에 대응.
Local Search + Global Search 이중 파이프라인 구조를 채택

04 Graph RAG — 파이프라인 구조

Indexing Pipeline (지식 그래프 구축)

Query Pipeline (검색 및 생성)

Local Search

특정 개체 중심 검색

1. 질문에서 엔터티 추출
2. KG에서 이웃 노드/관계 탐색
3. 관련 텍스트 청크 회수
4. LLM으로 답변 생성

Global Search

전체 코퍼스 수준 검색

1. 모든 커뮤니티 요약 사용
2. Map: 각 요약에서 부분 답변 생성
3. Reduce: 부분 답변 통합
4. 최종 LLM 응답 생성

Neo4j는 지식 그래프 저장소로서 KG 구축 단계와 Local Search 단계 모두에 활용됩니다.

05 실습 — Neo4j + LangChain Graph RAG

코드 예시 (Python)

```python
from langchain_community.graphs \
    import Neo4jGraph
from langchain_experimental.graph_\
    transformers import LLMGraphTransformer

# 1. 그래프 DB 연결
graph = Neo4jGraph(
  url="bolt://localhost:7687",
  username="neo4j", password="***")

# 2. 문서 → KG 변환
transformer = LLMGraphTransformer(llm=llm)
graph_docs = transformer.convert_to_graph_\
    documents(docs)

# 3. Neo4j에 저장
graph.add_graph_documents(graph_docs)

# 4. GraphRAG Chain 구성
chain = GraphCypherQAChain.from_llm(
    llm=llm, graph=graph, verbose=True)
```

구현 단계

1 **데이터 수집 및 청킹**
비정형 텍스트 로드
청크 크기: 512~1024 토큰

2 **엔터티·관계 추출**
LLM 기반 NER + 관계 추출
LLMGraphTransformer 활용

3 **Neo4j 저장 및 인덱싱**
노드/엣지 저장
Vector Index 병행 구축

4 **Cypher 생성 및 실행**
자연어 → Cypher 변환
결과 컨텍스트 회수

5 **LLM 최종 답변 생성**
그래프 컨텍스트 + 원문
RAG 기반 응답 합성

05 활용 사례 및 아키텍처 비교

도메인별 활용 사례

의료

- 질병-증상-치료 KG 구축
- 임상 의사결정 지원
- Drug-drug 상호작용 탐지

금융

- 기업-인물-거래 네트워크
- AML 이상거래 탐지
- 투자 관계 분석

법률

- 판례-법조문 관계 모델링
- 규제 준수 질의 응답
- 계약 리뷰 자동화

제조

- 부품-공급망 KG
- 결함 원인 추적
- 설비 예지 보전

기술 스택 선택 가이드

기준	Vector RAG	Graph RAG	Hybrid
의미 유사도 검색	우수	보통	우수
다중 홉 추론	미지원	우수	우수
구조화된 관계 탐색	미지원	우수	우수

온톨로지	도메인 지식을 형식적으로 모델링. RDF/OWL로 표준화된 스키마 정의
Neo4j	Property Graph 모델 기반 고성능 그래프 DB. Cypher로 직관적 탐색
지식 그래프	온톨로지(Schema) + 개체 데이터(Instance)의 통합. 대규모 팩트 네트워크
Graph RAG	벡터 검색의 한계를 KG 구조 탐색으로 보완. 다중 홉 추론·전역 질의 가능

참고자료: Edge, E. et al. (2024) From Local to Global: A Graph RAG Approach — arXiv:2404.16130 | Neo4j Documentation — neo4j.com/docs | Hogan, A. et al. (2021) Knowledge Graphs — ACM Comput. Surv.

지식 그래프

Knowledge Graph

개념 · 구조 · 활용 사례 · 장단점 완전 분석

지식 그래프

Definition & Core Concepts

지식 그래프(Knowledge Graph)는 현실 세계의 엔티티(개체)와 그 관계를 노드(Node)와 엣지(Edge)로 구조화한 그래프 형태의 데이터베이스

노드 (Node)

현실 세계의 개체 (사람, 장소, 제품, 개념 등)

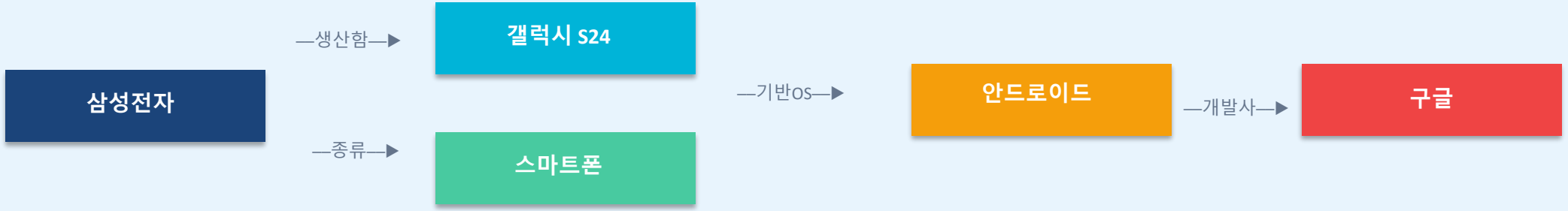
엣지 (Edge)

노드 간의 관계 (예: '삼성'은 '갤럭시'를 '생산한다').

속성 (Property)

엔티티와 관계에 대한 상세 정보 (설립연도, 국가 등)를 담습니다.

간단한 예시: 지식 그래프 구조



작동 원리 & 구조

How It Works

🎁 트리플 스토어 (Triple Store)

지식 그래프의 기본 저장 단위는 트리플(Triple):
주어(Subject) → 서술어(Predicate) → 목적어(Object)

예시: 이순신 → 직업 → 장군
예시: 서울 → 수도_of → 대한민국

📖 온톨로지 (Ontology)

개념과 관계의 체계적인 정의 (Schema)

- 클래스 계층 구조 정의
- 관계 유형 명시
- 데이터 타입 규칙 설정
- 추론(Inference) 가능케 함

🔎 SPARQL 쿼리 언어

```
SELECT ?name WHERE {
  ?person rdf:type :Engineer .
  ?person :worksAt :Kakao .
  ?person :name ?name .
}
```

🧠 추론 엔진 (Inference Engine)

명시되지 않은 관계를 자동 도출

- A가 B의 부모 → B는 A의 자녀 (대칭 추론)
- A → B → C이면 A → C (이행 추론)
- 새로운 사실 자동 생성 가능

주요 활용 사례

Key Use Cases

검색 엔진

- Google Knowledge Panel
- 연관 개체 자동 표시
- 맥락 기반 검색 결과
- 질문 응답 (QA) 시스템

추천 시스템

- Amazon 상품 추천
- Netflix 콘텐츠 추천
- 사용자 취향 맥락 분석
- 크로스셀링 최적화

의료 · 헬스케어

- 약물 상호작용 탐지
- 희귀질환 진단 보조
- 임상 의사결정 지원
- 의료 지식베이스 구축

금융 · 리스크

- 사기 거래 패턴 탐지
- 기업 관계망 분석
- AML(자금세탁방지)
- 신용 리스크 평가

AI / NLP

- LLM 지식 보강 (RAG)
- 개체명 인식(NER)
- 관계 추출 자동화
- 챗봇 도메인 지식 주입

제조 · 공급망

- 부품 계층 구조 관리
- 공급망 리스크 추적
- 디지털 트윈 연계
- 설비 고장 예측

장점 심층 분석

Advantages by Use Case

지식 그래프의 핵심 강점 — 사례별 분석

활용 분야	핵심 장점	구체적 효과
검색 엔진	맥락 이해 & 관계 기반 검색	클릭률 30~50% 향상, 제로클릭 답변 제공
추천 시스템	다차원 관계 반영한 정교한 추천	협업 필터링 대비 정확도 25% 개선
의료 진단	이종 데이터 통합 & 추론 가능	진단 오류 감소, 희귀질환 발견 가능성↑
금융 사기탐지	네트워크 패턴 분석 (그래프 탐색)	사기 적발률 60% 이상 향상
LLM / AI	환각(Hallucination) 감소	사실 기반 응답으로 신뢰도 향상
제조 / 공급망	복잡한 부품 계층 시각화	재고 비용 10~20% 절감

단점 & 한계점

Disadvantages & Challenges

⚠ 지식 그래프의 도전 과제 — 사례별 분석

구축 비용 & 시간

검색·의료

초기 구축에 도메인 전문가 필요, 대규모 그래프는 수년 소요

데이터 최신성 유지

금융·뉴스

지속적 업데이트 필요, 스테일 데이터 발생 가능성

스케일 & 성능

대규모 서비스

수십억 트리플에서 쿼리 성능 저하, 실시간 처리 한계

스키마 진화 어려움

제조·물류

온톨로지 변경 시 전체 데이터 재처리 필요

불완전성 문제

의료·법률

오픈 월드 가정으로 인한 지식 공백, 추론 오류 가능성

데이터 통합 복잡성

엔터프라이즈

다양한 소스 정합성 확보, 중복 엔티티 처리 어려움

실제 도입 성공 사례

Real-World Success Stories

Google

Knowledge Graph (2012~)

검색 결과 우측 패널에 구조화된 정보 제공

- 500억+ 엔티티, 5,000억+ 트리플
- 검색 만족도 30% 향상
- 자연어 질문 직접 답변 가능

Amazon

Product Graph

상품 속성& 관계 기반 추천/검색 엔진

- 3억+ 상품 간 관계 모델링
- 추천 정확도 35% 개선
- 장바구니 전환율 상승

Microsoft

Microsoft Graph

Office 365 사용자·조직·문서 관계망

- 수억 사용자 생산성 데이터 연결
- Teams/Outlook 인텔리전트 기능 기반
- 코파일럿 AI 컨텍스트 제공

AstraZeneca

Drug Knowledge Graph

약물-질환-유전자 관계 분석 플랫폼

- 신약 후보 물질 발굴 기간 40% 단축
- 약물 재창출(Drug Repurposing) 성공
- 부작용 예측 정확도 향상

Future Outlook & Conclusion

LLM + KG 융합

RAG(검색 증강 생성)으로 AI 환각 문제 해소, 사실 기반 AI 실현

⚡ 실시간 지식 그래프

스트리밍 데이터 처리로 동적 갱신, 금융·물류 실시간 적용

연합 지식 그래프

기업 간 프라이버시 보호하며 지식 공유, 의료·정부 데이터 연계

자동 지식 추출

NLP/ML로 비정형 텍스트에서 자동 관계 추출, 구축 비용 대폭 절감

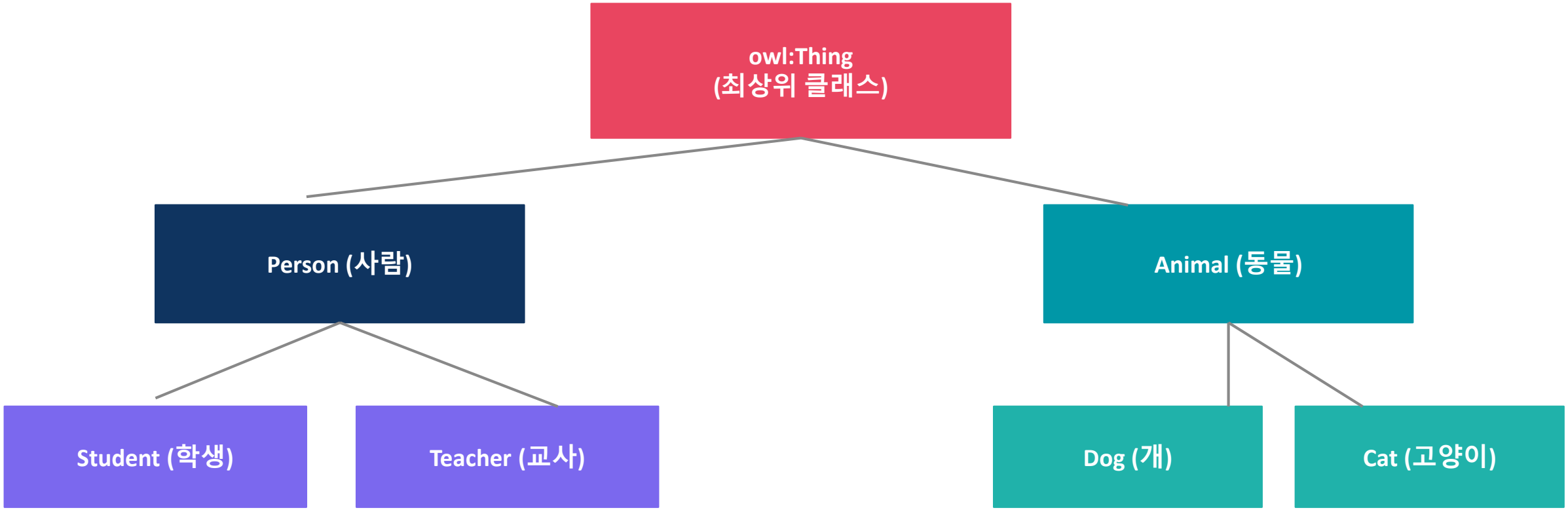
결론

지식 그래프는 단순한 데이터베이스를 넘어, AI 시대의 핵심 지식 인프라로 자리잡고 있습니다. 구축 비용과 복잡성이라는 단점에도 불구하고, 관계 기반 추론·맥락 이해·이종 데이터 통합이라는 강점은 검색·의료·금융·AI 전 산업에서 그 가치를 입증하고 있습니다.

온톨로지와 LPG

Labeled Property Graph

OWL 클래스 계층과 프로퍼티



◈ Object Property
개체 간의 관계 (예: hasStudent)

◈ Datatype Property
데이터값과의 관계 (예: hasAge 25)

LPG (Labeled Property Graph)

● 노드 (Node)

- 개체/인스턴스를 표현
- 하나 이상의 레이블 보유
- 속성(key-value) 저장

→□ 엣지 (Edge)

- 노드 간 관계를 표현
- 방향성 있음
- 하나의 레이블 보유

🏷 레이블 (Label)

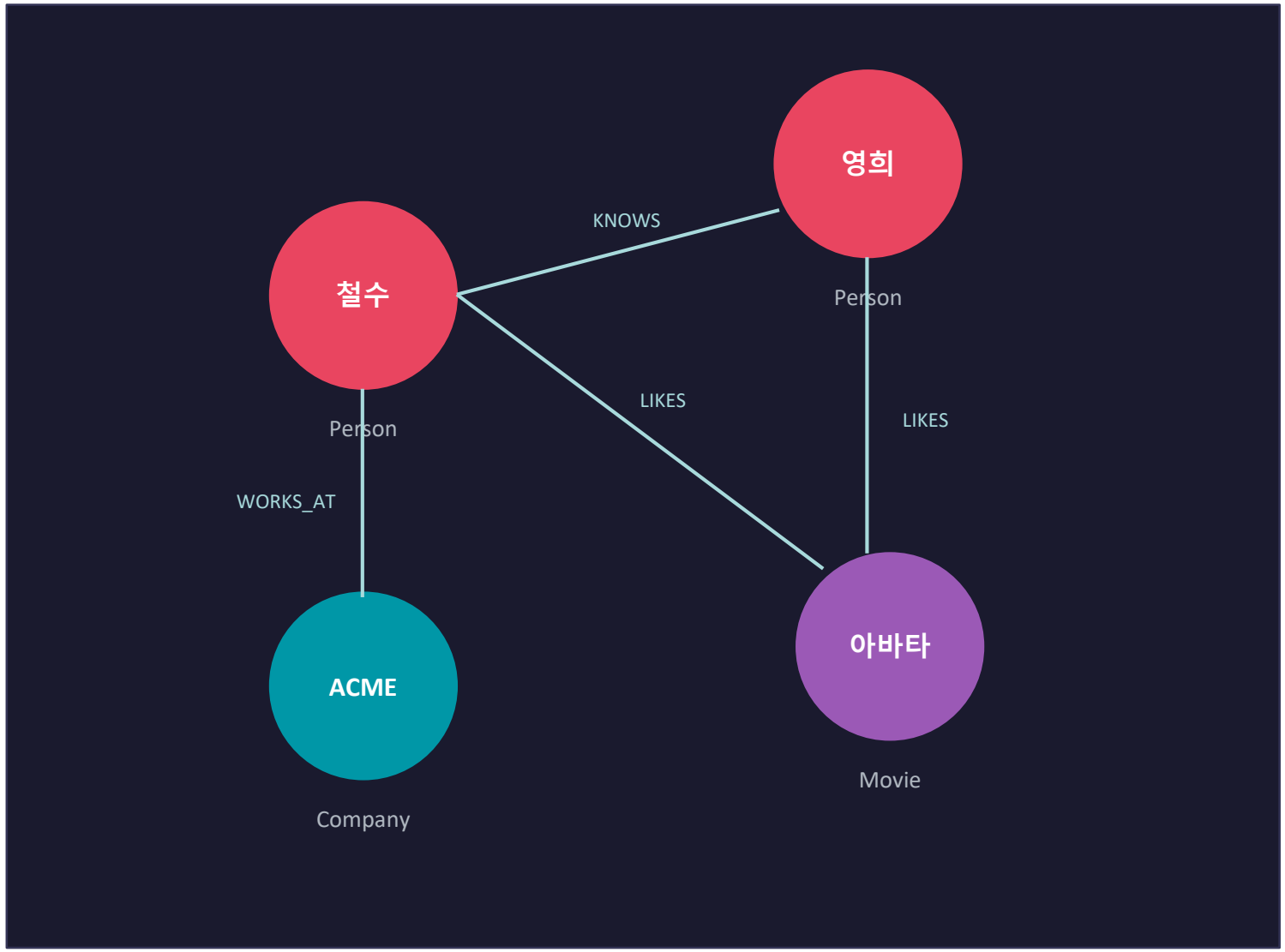
- 노드/엣지의 타입 구분
- 복수 레이블 가능
- 분류 및 인덱싱 역할

📝 속성 (Property)

- key-value 형태
- 노드와 엣지 모두 보유
- 다양한 데이터 타입 지원

```
(:Person {name:"철수", age:25}) -[:KNOWS {since:2020}]-> (:Person {name:"영희"})
```

LPG 그래프 구조 예시



노드 속성 예시

```
(:Person)
  name: "철수"
  age: 25
  email: "cs@ex.com"

[:KNOWS]
  since: 2020
  weight: 0.9
```

온톨로지 vs LPG 비교

비교 항목	온톨로지 (RDF/OWL)	LPG (Neo4j 등)
데이터 모델	트리플 (S-P-O)	노드 + 엣지 + 속성
표준화	W3C 국제 표준	사실상 표준
추론 지원	내장 추론 가능	직접 구현 필요
스키마	엄격한 타입 시스템	유연한 스키마리스
쿼리 언어	SPARQL	Cypher / Gremlin
성능	대규모 추론 시 부담	빠른 그래프 탐색
학습 난이도	높음 (논리 기반)	상대적으로 낮음

Neo4j Cypher 기초 문법

노드 생성 (CREATE)

```
CREATE (n:Person {name:"철수", age:25})
CREATE (m:Person {name:"영희", age:23})
```

관계 생성

```
MATCH (a:Person {name:"철수"}), (b:Person {name:"영
희"})
CREATE (a)-[:KNOWS {since:2020}]->(b)
```

노드 조회 (MATCH)

```
MATCH (n:Person)
WHERE n.age > 20
RETURN n.name, n.age
```

경로 탐색

```
MATCH p=(a:Person)-[:KNOWS*1..3]->(b:Person)
RETURN p
```

Cypher 심화: MERGE / DELETE / 집계

MERGE (없으면 생성)

```
MERGE (n:Person {name:"민준"})
ON CREATE SET n.created = timestamp()
ON MATCH SET n.updated = timestamp()
```

SET / DELETE

```
MATCH (n:Person {name:"철수"})
SET n.city = "Seoul"

MATCH (n:Person {name:"철수"})
DETACH DELETE n
```

집계 함수

```
MATCH (n:Person)
RETURN
  count(n) AS total,
  avg(n.age) AS avgAge,
  max(n.age) AS maxAge
```

WITH / ORDER BY

```
MATCH (p:Person)-[:KNOWS]->(friend)
WITH p, count(friend) AS cnt
WHERE cnt >= 2
RETURN p.name, cnt
ORDER BY cnt DESC
```

온톨로지 → LPG 변환 전략

owl:Class	→	노드 레이블	`Person → (:Person)`
owl:Individual	→	노드 인스턴스	`:철수 → (철수:Person)`
owl:ObjectProperty	→	엣지 (관계)	`knows → -[:KNOWS]->`
owl:DatatypeProperty	→	노드 속성	`hasAge 25 → {age: 25}`
rdfs:subClassOf	→	레이블 복수 부여	`Student ⊂ Person → (:Student:Person)`

실습 - Python으로 RDF → Neo4j 변환

```python
from rdflib import Graph
from neo4j import GraphDatabase

# RDF 그래프 로드
g = Graph().parse("ontology.ttl", format="turtle")

# Neo4j 연결
driver = GraphDatabase.driver(
    "bolt://localhost:7687",
    auth=("neo4j", "password"))

# 트리플을 Cypher로 변환
with driver.session() as session:
    for s, p, o in g:
        session.run("""
            MERGE (a {uri: $subj})
            MERGE (b {uri: $obj})
            MERGE (a)-[r {uri: $pred}]->(b)
        """, subj=str(s), pred=str(p), obj=str(o))
```

프로젝트 : 영화 지식 그래프

1

온톨로지 설계

Movie, Person, Genre 클래스
ACTED_IN, DIRECTED 관계 정의

2

Neo4j 스키마 매핑

OWL 클래스 → 노드 레이블
ObjectProperty → 엣지 타입

3

데이터 입력

Cypher CREATE/MERGE로
영화, 배우, 감독 입력

4

쿼리 실습

특정 배우 출연 영화 조회
장르별 평점 집계

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)<-[:DIRECTED]-(d:Director) RETURN a.name, m.title, d.name
```

실습 전체 코드 : 영화 지식 그래프

```
// 1. 데이터 생성
CREATE (m1:Movie {title:"아바타", year:2009, rating:7.8})
CREATE (m2:Movie {title:"인터스텔라", year:2014, rating:8.6})
CREATE (a1:Actor {name:"샘 워딩턴"})
CREATE (d1:Director {name:"제임스 카메론"})
CREATE (d2:Director {name:"크리스토퍼 놀란"})

// 2. 관계 생성
MATCH (a:Actor {name:"샘 워딩턴"}), (m:Movie {title:"아바타"})
CREATE (a)-[:ACTED_IN {role:"제이크"}]->(m)

MATCH (d:Director {name:"제임스 카메론"}), (m:Movie {title:"아바타"})
CREATE (d)-[:DIRECTED]->(m)

// 3. 조회 - 영화별 감독과 배우
MATCH (d:Director)-[:DIRECTED]->(m:Movie)<-[:ACTED_IN]-(a:Actor)
RETURN m.title, d.name AS director, collect(a.name) AS actors
```

실세계 활용 사례

Google · Knowledge Graph

검색 결과에 지식 패널 제공
약 500억 개 노드 보유

Amazon · Product Graph

상품 추천, 연관 상품 탐색
고객 구매 패턴 분석

의생명 · Bio Ontology

Gene Ontology, SNOMED CT
신약 개발 지식 관리

금융 · Risk Graph

사기 탐지, 자금 세탁 추적
거래 관계 시각화

AI 챗봇 · KG + LLM

RAG(검색 증강 생성)
정확도 향상, 환각 감소

엔터프라이즈 · Enterprise KG

HR, 조직도, 역량 관리
내부 검색 시스템

도구 및 생태계

온톨로지 편집기

- Protégé (Stanford 무료)
- TopBraid Composer
- WebProtégé (온라인)

RDF 라이브러리

- Python: rdflib
- Java: Jena
- JavaScript: rdf-ext

추론 엔진

- HermiT, Pellet
- RDFox
- Stardog (상용)

그래프 DB (LPG)

- Neo4j
- Amazon Neptune
- ArangoDB (멀티모델)

쿼리 언어

- SPARQL → RDF/온톨로지용
- Cypher → Neo4j용
- Gremlin → Apache TinkerPop

시각화 도구

- Neo4j Browser
- Gephi
- GraphXR

참고 자료

Neo4j GraphRAG Python Package	*github.com/neo4j/neo4j-graphrag-python*
Microsoft GraphRAG (논문 + 코드)	*github.com/microsoft/graphrag*
LangChain Neo4j Integration	*python.langchain.com/docs/integrations/graphs/neo4j_cypher*
LlamaIndex KnowledgeGraphIndex	*docs.llamaindex.ai/en/stable/examples/index_structs/knowledge_graph*
Neo4j AuraDB Free Tier	*neo4j.com/cloud/platform/aura-graph-database*